

Vulnerability Assessment Report

Fictional Food Delivery / E-Commerce Application Assessment — Academic Sample

1. Executive Summary

A vulnerability assessment was conducted against a fictional food-delivery application to identify security weaknesses that could affect customer data, account security, and business operations. The assessment identified two representative application-level issues: SQL Injection and Insecure Direct Object Reference (IDOR). If present in a production system, these weaknesses could expose sensitive records or allow unauthorized access to other users' resources. The findings should be validated by the development/security team before production deployment. Recommended actions include parameterized database queries, strict server-side authorization checks, secure testing, and regression testing after remediation.

2. Findings Summary

ID	Finding	Risk	Potential Impact
F-01	SQL Injection	High	Unauthorized database access or manipulation; possible exposure of customer/order data.
F-02	Insecure Direct Object Reference (IDOR)	High	Unauthorized access to another user's resource when server-side authorization is missing.

3. Detailed Findings

F-01 — SQL Injection

Description: SQL Injection occurs when untrusted input is incorporated into database queries without adequate parameterization. In a fictional shopping application, an affected search, login, or order-management function could allow unintended database operations.

Risk: High — potential impact includes unauthorized disclosure or modification of application data, depending on database privileges and application architecture.

Sample evidence (fictional): GET /search?q=[test-input] → application returned a database error pattern. This is illustrative evidence for the academic report, not a claim about a real service.

Recommendation: Use parameterized queries/prepared statements for all database operations. Validate input according to business requirements, use least-privilege database accounts, avoid exposing database errors to users, and add automated security/regression tests for database-facing endpoints.

F-02 — Insecure Direct Object Reference (IDOR)

Description: IDOR occurs when an application accepts a user-controlled object identifier but does not verify that the authenticated user is authorized to access that object. For example, an order endpoint might expose an order belonging to a different account when only the identifier is changed.

Risk: High — successful exploitation could expose another customer's order, address, profile, or other protected information, depending on the affected endpoint.

Sample evidence (fictional): GET /api/orders/1002 → 200 OK for a resource not owned by the test account. This is illustrative evidence for the academic report.

Recommendation: Enforce authorization on the server for every object request. Derive the user's identity from the authenticated session/token, verify ownership or permitted role before returning the object, and add authorization tests covering horizontal and vertical access-control boundaries.

4. Reporting Notes / Appendix

The structure follows OWASP Web Security Testing Guide reporting guidance: an executive summary for business readers, technical findings with risk and remediation information, and supporting methodology/evidence. OWASP notes that an assessment is a point-in-time activity and does not guarantee that every possible issue has been identified.

Scope: Fictional application used for academic demonstration. Findings and evidence above are illustrative and were not obtained from Flipkart, Myntra, Zomato, or another real company's systems.

Reference: OWASP Web Security Testing Guide — Reporting Structure.